SIRAV Informe ejecutivo

Sistema Integral de Reservas y Alquiler de Vehículos
Bases de Datos 2 · Universidad El Bosque
Implementación y alcance verificado · 21 de octubre de 2026

Problema

Una empresa pequeña de alquiler necesita coordinar vehículos, clientes y fechas sin asignar el mismo vehículo a dos operaciones superpuestas. Los registros manuales separan la entrega, las condiciones del automóvil, la devolución y los cobros, lo que dificulta detectar conflictos y reconstruir lo ocurrido.

Solución implementada

SIRAV integra el ciclo de reserva, confirmación, entrega y devolución en una aplicación web. La base relacional controla las reglas críticas; las inspecciones conservan listas variables de daños y evidencias en MongoDB. Administrador, operador y cliente acceden a capacidades distintas mediante autenticación JWT y autorización en el servidor.

Componente	Responsabilidad
React	Catálogo, formularios operativos, historial, inspecciones y reportes.
Spring Boot y Java	API REST, validación, JWT, permisos, transacciones y coordinación entre bases.
PostgreSQL y PL/pgSQL	Trece entidades del dominio, procedimientos, funciones, triggers, excepciones e integridad.
MongoDB	Documentos de inspección y bitácora; CRUD, búsqueda compleja y agregaciones.

Alcance del prototipo

Se implementan tarifas por vigencia, mantenimiento, cotización, reservas sin cruces, cancelaciones, inspecciones, cargos por retraso y pagos simulados. El documento definitivo excluye Oracle: se utiliza PL/pgSQL como mecanismo equivalente a PL/SQL. No se incorporan pagos reales ni almacenamiento de fotografías; las evidencias se registran por referencia.

Diseño y decisiones técnicas

Modelo relacional y reglas

El esquema conserva las 13 entidades suministradas: Usuario, Rol, Usuario_Rol, Cliente, Categoria_Vehiculo, Vehiculo, Tarifa, Reserva, Alquiler, Pago, Cargo_Adicional, Mantenimiento y Auditoria_Reserva. Incluye relaciones 1:1, 1:N y N:M y separación de dependencias hasta 3FN. Los totales y la disponibilidad se calculan, en lugar de duplicarse en Reserva.

Los intervalos se representan como [inicio, fin). Una restricción de exclusión por vehículo impide reservas solapadas en estado SOLICITADA, CONFIRMADA o ACTIVA. Los procedimientos bloquean el vehículo antes de validar cruces con mantenimientos. La fecha fin debe ser posterior al inicio y la tarifa debe cubrir todo el período.

Lógica de negocio y atomicidad

Cinco procedimientos gestionan crear, confirmar, cancelar, iniciar y cerrar. Tres funciones calculan disponibilidad, costo base y retraso. Los triggers validan transiciones, mantenimiento, auditoría y estado operativo. La aplicación llama a estos objetos con parámetros y confirma la transacción completa mediante Spring; un error revierte los cambios. El repositorio incluye un ejemplo ejecutable de COMMIT y ROLLBACK.

Cada día calendario de retraso genera un cargo del 50% de la tarifa diaria. Al cerrar, el sistema genera un pago SIMULADO con el costo base y los cargos. El kilometraje no puede disminuir y el combustible debe estar entre 0 y 100. La entrega y la devolución requieren una inspección, aunque no tenga daños.

Consistencia documental

Una tabla técnica outbox guarda el documento de inspección en la misma transacción del alquiler. Un publicador lo envía a MongoDB y reintenta si el motor no está disponible. El UUID del evento permite una escritura idempotente; un reintento no duplica ni sobrescribe una inspección corregida. La interfaz informa cuando una inspección está pendiente de sincronización.

Seguridad

Las contraseñas usan BCrypt. Los JWT expiran y se revocan al cerrar sesión o cambiar permisos. Cada petición comprueba la cuenta activa y la versión del token. El cliente solo consulta sus propias operaciones. El rol SQL de la API no escribe directamente reservas, alquileres, pagos ni auditoría; ejecuta procedimientos controlados. Los secretos se configuran fuera de Git.

Resultados y validación

La verificación funcional se realizó en Windows con PostgreSQL 18 local, MongoDB 8.0.5 y JDK 23 compilando para Java 21. La configuración de despliegue utiliza PostgreSQL 17 y Java 21 en contenedores; el despliegue Docker y la VM Linux requieren una ejecución adicional en el entorno de entrega.

Pruebas automatizadas

Comprobación	Evidencia de implementación
Backend	Pruebas de autorización, exportación segura, ciclo completo, concurrencia, permisos SQL, reportes y CRUD MongoDB. Los resultados de cada ejecución se conservan en Surefire y se resumen en docs/PRUEBAS.md.
Atomicidad	Una devolución con kilometraje inválido es rechazada y no crea pago; la devolución válida cierra el alquiler y genera las dos inspecciones del ciclo.
Concurrencia	Dos solicitudes simultáneas para el mismo vehículo y período producen exactamente una reserva aceptada.
MongoDB	Consulta de severidad alta, corrección versionada, rechazo de versión obsoleta, anulación lógica y tres agregaciones.
Frontend	Pruebas de autenticación y registro, compilación de producción y pruebas de navegación en navegador de escritorio y móvil.

Consultas complejas

Los seis reportes cubren ocupación mensual con calendario y días distintos; ingresos por categoría y mes; ranking de los vehículos por categoría con funciones de ventana; actividad y tasa de cancelación de clientes; devoluciones tardías con HAVING; y disponibilidad con anti joins. Las agregaciones documentales calculan promedios por categoría, zonas de daño por mes e incidencias por vehículo.

Límites de los resultados

Los datos son de demostración y los pagos no representan transacciones financieras. Las pruebas no equivalen a una auditoría de seguridad ni a una certificación de rendimiento. No se afirma el objetivo de dos segundos en una VM sin medirlo allí. La actualización de inspecciones es eventualmente consistente y la interfaz puede mostrar una sincronización pendiente durante unos segundos.

Gestión y cierre de entrega

Organización propuesta

Se entrega un plan híbrido de cuatro semanas, una EDT con estimación de 124 horas, hitos, matriz RACI para tres integrantes, ocho riesgos con probabilidad e impacto, historias con aceptación y una plantilla de trazabilidad. La organización responde al documento definitivo, que exige tres estudiantes; los nombres y responsabilidades reales deben validarse por el equipo.

Despliegue y evidencia académica

El repositorio incluye Docker Compose, Dockerfiles, creación de usuarios con privilegios limitados, configuración por entorno, respaldo y guía de instalación en Ubuntu Server dentro de una máquina virtual. El guion distribuye una demostración de 13 minutos y 30 segundos entre los integrantes.

Siguen requiriendo ejecución personal del equipo: crear y documentar la VM Linux, validar su acceso desde el anfitrión, conservar evidencias de respaldo y restauración, generar el historial colaborativo en Git y grabar el video. Los modelos deben revisarse en la herramienta aceptada por el docente. No se presentan planes ni guías como pruebas de actividades que no se han realizado.

Lecciones y evolución

La prevención de cruces pertenece a la base y debe comprobarse con concurrencia real. La combinación de dos motores necesita un mecanismo explícito de recuperación; no basta con escribir secuencialmente en ambos. Las consultas financieras deben agregar cada relación antes de unirla, para evitar totales duplicados. Una siguiente iteración puede incorporar recuperación de contraseña, almacenamiento de evidencias, medición de rendimiento en la VM y monitoreo de eventos pendientes.

Conclusión

La implementación conecta la aplicación web con los dos motores y concentra las reglas transaccionales en PL/pgSQL. El código, los scripts, los modelos y los manuales permiten reproducir el prototipo y preparar la sustentación. La entrega académica debe completarse con las evidencias reales de infraestructura y participación del equipo.

Referencias principales

Elmasri, R., y Navathe, S. B. (2016). Fundamentals of Database Systems. 7.ª ed. Pearson.

Silberschatz, A., Korth, H. F., y Sudarshan, S. (2019). Database System Concepts. 7.ª ed. McGraw-Hill.

Bradshaw, S., Brazil, E., y Chodorow, K. (2019). MongoDB: The Definitive Guide. 3.ª ed. O'Reilly.

Gray, J. (1981). The transaction concept: virtues and limitations. VLDB, 144-154.